

# Tokens: Chopping Text into Pieces

How text becomes numbers a model can read · video 2:53

This lesson answers a question episode 1 skipped: what exactly goes into the model? A neural network only works with numbers, so text is first chopped into **tokens**, usually pieces of words, and each token is replaced by a number, its **ID**. The pieces are not chosen by hand: an algorithm called **byte pair encoding** learns them by counting which letters appear side by side.

## How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

## Concepts in this lesson

| CONCEPT                                             | VIDEO SEGMENT | QUESTIONS | MASTERED                 |
|-----------------------------------------------------|---------------|-----------|--------------------------|
| 1. Text in, numbers out: the tokenizer              | ▶ 0:08-0:29   | 3         | <input type="checkbox"/> |
| 2. Two extremes: characters or words                | ▶ 0:29-0:58   | 4         | <input type="checkbox"/> |
| 3. Subword tokens: the middle road                  | ▶ 0:58-1:16   | 4         | <input type="checkbox"/> |
| 4. Byte pair encoding: merge the most frequent pair | ▶ 1:16-1:50   | 4         | <input type="checkbox"/> |
| 5. BPE training in code                             | ▶ 1:50-2:01   | 3         | <input type="checkbox"/> |
| 6. Vocabulary and token IDs                         | ▶ 2:01-2:22   | 3         | <input type="checkbox"/> |
| 7. The model never sees letters                     | ▶ 2:22-2:50   | 4         | <input type="checkbox"/> |

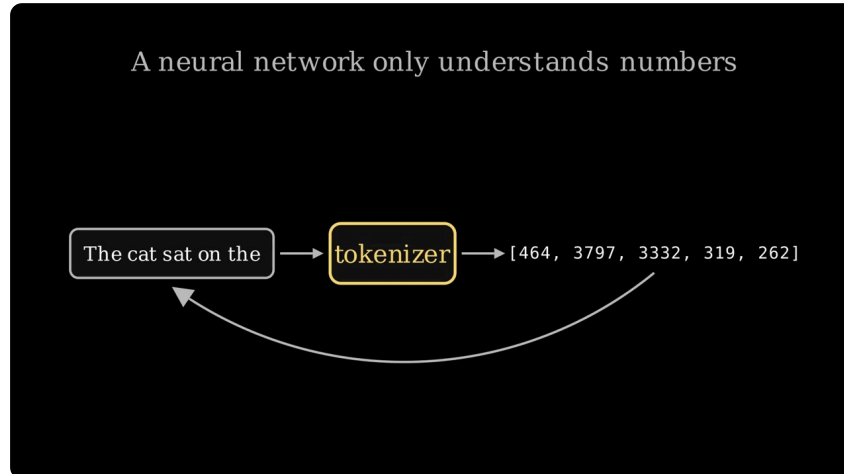
**Before you start:** this lesson builds on episode 1 (the predict → pick → append loop). No maths is needed. Concept 5 reads a short Python loop; if Python is new to you, Foundations F11 (NumPy in Three Minutes) helps. A note on notation: ◻ marks a space, so ◻cat is “cat” with a space in front.

## CONCEPT 1 OF 7

# Text in, numbers out: the tokenizer

▶ Video 0:08–0:29

Episode 1 said an LLM predicts the next word. That was a small lie: it predicts the next **token**. The reason is that a neural network only understands **numbers**. So before anything else, text must become a list of numbers, and the model's output must be turned back into text.



▶ 0:28 The tokenizer turns text into a list of numbers, and numbers back into text (the curved arrow).

### KEY IDEA

The **tokenizer** converts text into a list of token **IDs** (“*The cat sat on the*” → [464, 3797, 3332, 319, 262]) and converts IDs back into text. The model itself only ever sees the numbers.

### Check 1: Text in, numbers out: the tokenizer

Answer all 3 questions before looking at the answers. Pass mark: all correct.

#### 1.1 TRUE OR FALSE

“An LLM reads the letters of your text directly.”

True ☐ False ☐

#### 1.2 MULTIPLE CHOICE

What does a tokenizer do?

- A. Corrects spelling before the model sees the text
- B. Turns text into a list of numbers, and numbers back into text
- C. Predicts the next token
- D. Picks the most likely word from the probabilities

#### 1.3 SHORT ANSWER

The model predicts that the next token is ID 262. What has to happen before you see anything on screen? What appears?

---

---

## CONCEPT 2 OF 7

# Two extremes: characters or words

▶ Video 0:29-0:58

The simplest tokenizer uses **one token per character**: a tiny vocabulary (a few hundred symbols), but long sequences, and a single letter like *t* carries almost no meaning. The other extreme, **one token per word**, gives short sequences, but the vocabulary explodes: every name, typo and bit of slang needs an entry, and a word never seen in training, like *catfluencer*, can't be read at all.

### Idea 1: one token per character

T h e \_ c a t \_ s a t \_ o n \_ t h e  
84 104 101 32 99 97 116 32 115 97 116 32 111 110 32 116 104 101

✓ vocabulary: a few hundred symbols

✗ 18 tokens for 5 words

t meaning?

▶ 0:42 Characters: 18 tokens for 5 words.

### Idea 2: one token per word

The cat sat on the

✓ short sequences: 5 tokens

✗ vocabulary: 500,000+ words?

catfluencer → [UNKNOWN] ✗

▶ 0:57 Words: a new word becomes [UNKNOWN].

## KEY IDEA

Characters: **small vocabulary, long sequences**. Words: **short sequences, huge vocabulary**, and unknown words are lost. Neither extreme works well.

## Check 2: Two extremes: characters or words

Answer all 4 questions before looking at the answers. Pass mark: all correct.

### 2.1 CALCULATION

How many tokens is “The cat sat on the mat” with one token per character (spaces count)? Per word?

### 2.2 MULTIPLE CHOICE

A word-level tokenizer meets *rizzmaster*, a word that was not in its training text. What happens?

- A. It splits it into *rizz* + *master*
- B. It becomes [UNKNOWN]: the model can't read it
- C. It adds the word to its vocabulary on the fly
- D. It spells it out letter by letter

### 2.3 TRUE OR FALSE

“Character tokens can spell any text, even words never seen in training.”

True ☐ False ☐

### 2.4 MULTIPLE CHOICE

Which problem belongs to character tokens, not to word tokens?

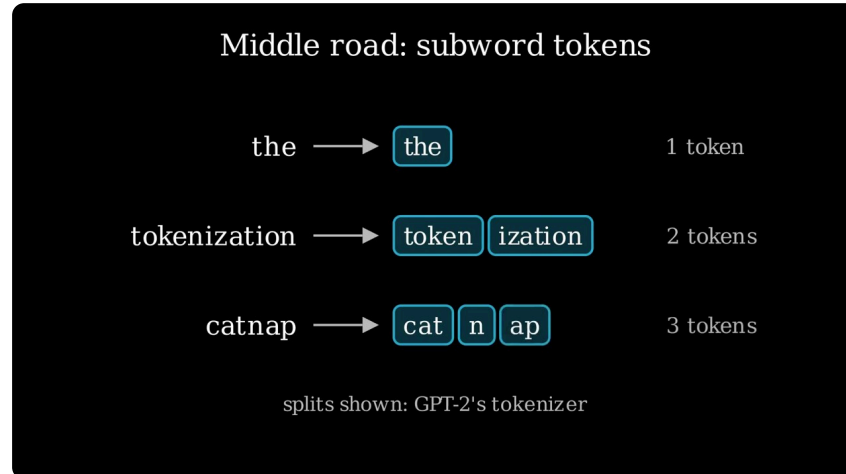
- A. The vocabulary needs hundreds of thousands of entries
- B. Typos and new slang become unknown
- C. Sequences are long and each token carries almost no meaning
- D. Every name needs its own token

## CONCEPT 3 OF 7

# Subword tokens: the middle road

▶ Video 0:58-1:16

Modern LLMs use **subword tokens**. Common words get a single token; rarer words are built from pieces. In GPT-2's tokenizer, *the* is one token, *tokenization* is two (*token* + *ization*) and *catnap* is three (*cat* + *n* + *ap*).



▶ 1:15 Subword tokens in GPT-2: a common word is one token, rarer words are built from pieces.

### KEY IDEA

**Subword tokens** get the best of both extremes: common words = one token, rare words = several pieces. Sequences stay fairly short, the vocabulary stays manageable, and a brand-new word can still be read as a sequence of pieces.

### Check 3: Subword tokens: the middle road

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 3.1 SHORT ANSWER

Why does GPT-2 split *tokenization* into two pieces but keep *the* as one token?

#### 3.2 MULTIPLE CHOICE

GPT-2 has no token for the made-up word *catfluencer*. What does its tokenizer do with it?

- A. Outputs [UNKNOWN]
- B. Builds it from smaller pieces
- C. Refuses the whole input
- D. Replaces it with the closest known word

#### 3.3 CALCULATION

Tokenized one at a time, how many GPT-2 tokens do *the*, *tokenization* and *catnap* take in total? How many tokens would a character tokenizer need for the same three words?

#### 3.4 TRUE OR FALSE

"With subword tokens, every English word is exactly one token."

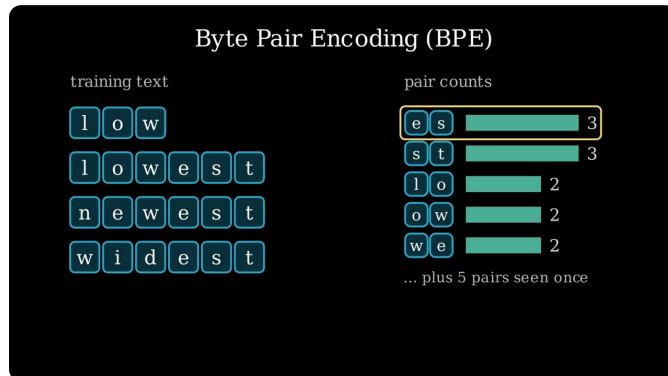
True ☐ False ☐

## CONCEPT 4 OF 7

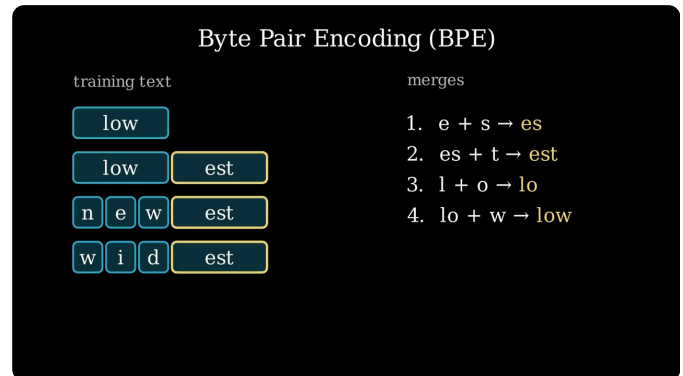
# Byte pair encoding: merge the most frequent pair

▶ Video 1:16-1:50

How are the pieces chosen? The most popular method is **byte pair encoding (BPE)**. Start with single characters. Count every pair of neighbours in a pile of training text. **Merge the most frequent pair** into a brand-new token. Then count again, and repeat.



▶ 1:28 Start with single characters and count every pair of neighbours. e+s is a top pair (3 times).



▶ 1:49 After four merges the tokenizer has found the word low and the ending est.

In the toy text *low, lowest, newest, widest*, e+s appears 3 times, as often as any pair, so it merges first into es. Next *es+t → est*, then *l+o → lo*, then *lo+w → low*. Nobody told it about words: it found *low* and *est* just by counting.

### KEY IDEA

BPE repeats one step: **count neighbouring pairs, merge the most frequent one into a new token**. Each merge adds one token to the vocabulary.

### Check 4: Byte pair encoding: merge the most frequent pair

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 4.1 PUT IN ORDER

Put the steps of BPE training in order: *merge the most frequent pair* · *start with single characters* · *repeat* · *count every pair of neighbours*.

#### 4.2 SHORT ANSWER

New training text: *hug, pug, hugs, pun*. Which pair is merged first, and how many times does it appear?

#### 4.3 SHORT ANSWER

Apply the video's four merges, in order (e+s, es+t, l+o, lo+w), to the new word *slowest*. Which tokens do you get?

#### 4.4 TRUE OR FALSE

"Once e and s are merged into es, the token e is no longer needed."

True ☐ False ☐

## CONCEPT 5 OF 7

# BPE training in code

▶ Video 1:50-2:01

In code, training a tokenizer is a short loop. `words` is the training text split into characters. Each round counts the pairs, picks the most frequent one, merges it everywhere and records the merge.



▶ 2:00 Training a tokenizer: a short loop, repeated tens of thousands of times.

```
def train_bpe(words, num_merges):
    merges = []
    for _ in range(num_merges):
        pairs = count_pairs(words)           # neighbour pairs -> count
        best = max(pairs, key=pairs.get)     # most frequent pair
        words = merge_pair(words, best)      # glue it everywhere
        merges.append(best)
    return merges
```

### KEY IDEA

One loop iteration = **one merge = one new token**. `num_merges` sets how many tokens are added to the starting characters: tens of thousands for a real tokenizer.

### Check 5: BPE training in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

#### 5.1 MULTIPLE CHOICE

Why does the loop call `count_pairs` again in every round, instead of counting once at the start?

- A. To make training slower but safer
- B. Because each merge changes the words, so the pair counts change
- C. Because `max` can only be used once per list
- D. It doesn't need to; counting once gives the same result

#### 5.2 CALCULATION

The toy text *low, lowest, newest, widest* uses 9 different letters. After the 4 merges of concept 4, how many different tokens are in the vocabulary?

.....

### 5.3 HANDS-ON CODE

**Try it yourself.** Below are `merge_pair` and `train_bpe` for the toy text. (a) Write `count_pairs(words)`, which returns a `Counter` of every neighbouring pair (hint: `zip(w, w[1:])`). How many different pairs does it find, and what is the count of `('e', 's')`? (b) What does `train_bpe(words, 4)` return? (c) Run `train_bpe(words, 5)`. What is the fifth merge, and why is that choice somewhat arbitrary?

```
from collections import Counter

def merge_pair(words, pair):
    out = []
    for w in words:
        new, i = [], 0
        while i < len(w):
            if i + 1 < len(w) and (w[i], w[i + 1]) == pair:
                new.append(w[i] + w[i + 1]) # glue the pair
                i += 2
            else:
                new.append(w[i])
                i += 1
        out.append(new)
    return out

def train_bpe(words, num_merges):
    merges = []
    for _ in range(num_merges):
        pairs = count_pairs(words)
        best = max(pairs, key=pairs.get)
        words = merge_pair(words, best)
        merges.append(best)
    return merges

words = [list(w) for w in ["low", "lowest", "newest", "widest"]]
```

## CONCEPT 6 OF 7

# Vocabulary and token IDs

▶ Video 2:01-2:22

All the merges together give a **vocabulary**. GPT-2 has about fifty thousand tokens (50,257); newer models use 100,000 or more. Each token gets an **ID**, and a sentence becomes a short list of numbers.

| vocabulary |                   |
|------------|-------------------|
| ID         | token             |
| 262        | <code>▁the</code> |
| 319        | <code>▁on</code>  |
| 464        | <code>The</code>  |
| 3332       | <code>▁sat</code> |
| 3797       | <code>▁cat</code> |
| 9246       | <code>cat</code>  |
| ⋮          |                   |

GPT-2: 50,257 tokens  
newer models: 100,000+

▶ 2:11 The merges give a vocabulary; each token gets an ID. GPT-2 has 50,257 tokens.

|                  |                   |                   |                  |                   |
|------------------|-------------------|-------------------|------------------|-------------------|
| <code>The</code> | <code>▁cat</code> | <code>▁sat</code> | <code>▁on</code> | <code>▁the</code> |
| 464              | 3797              | 3332              | 319              | 262               |
|                  |                   |                   |                  |                   |
|                  | <code>▁cat</code> | →                 | 3797             |                   |
|                  | <code>cat</code>  | →                 | 9246             |                   |

▶ 2:21 Spaces are part of the tokens: `▁cat` (3797) and `cat` (9246) are different tokens.

Notice that **spaces are part of the tokens**. `▁cat` (with a space in front, ID 3797) and `cat` (no space, ID 9246) are different tokens, so “The cat sat on the” is really `The` · `▁cat` · `▁sat` · `▁on` · `▁the`.

### KEY IDEA

Text → tokens → **IDs**, one number per token from a fixed vocabulary. A leading space is part of a token, so the same word can have two IDs.

### Check 6: Vocabulary and token IDs

Answer all 3 questions before looking at the answers. Pass mark: all correct.

#### 6.1 SHORT ANSWER

Use the vocabulary table in the left figure to decode [464, 3332, 319, 262, 9246]. Watch the spaces!

#### 6.2 CALCULATION

GPT-2's tokenizer starts from 256 basic symbols (one per possible byte), makes 50,000 merges, and adds one special end-of-text token. How big is its vocabulary?

#### 6.3 MULTIPLE CHOICE

In GPT-2, why does `cat` get a different ID in “cat food” than in “my cat”?

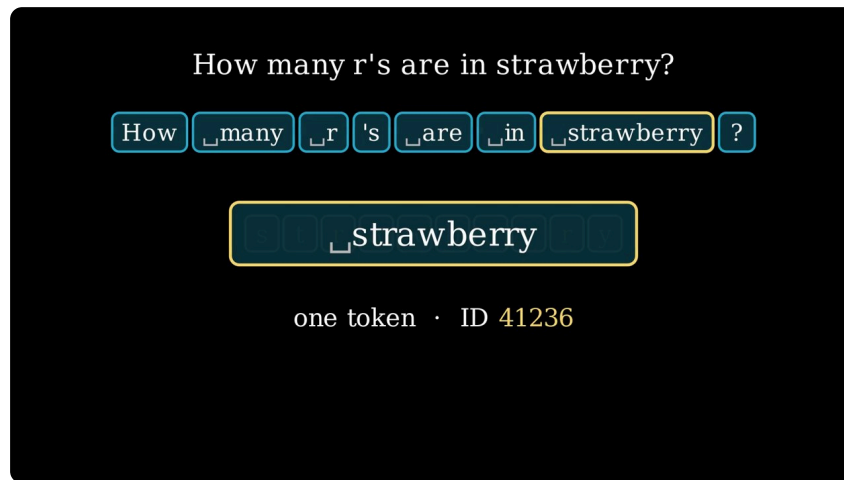
- A. IDs are assigned at random each time
- B. In “my cat” the token is `▁cat`, with a space in front
- C. IDs depend on the word's position in the sentence
- D. The tokenizer ignores the second word of every text

## CONCEPT 7 OF 7

# The model never sees letters

▶ Video 2:22-2:50

Tokens explain some odd behaviour. Ask a model how many r's are in *strawberry* and it may stumble. That's because it never sees the letters: to GPT-2, `▮strawberry` is **one single token**, ID 41236.



▶ 2:35 To GPT-2, `▮strawberry` is one single token, ID 41236.

### KEY IDEA

The model sees **token IDs, not letters**: the letters inside a token are hidden from it. And an ID like 3797 is **just a label** that says nothing about what a cat is; episode 3 turns each token into something richer, a vector.

### Check 7: The model never sees letters

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 7.1 MULTIPLE CHOICE

Why can a model miscount the r's in *strawberry*?

- A. It never saw the word during training
- B. It sees `▮strawberry` as one ID, not as separate letters
- C. Its vocabulary has no letter r
- D. The question has too many tokens

#### 7.2 CALCULATION

"How many r's are in *strawberry*?" has 31 characters. How many GPT-2 tokens is it? (Count them in the figure.)

#### 7.3 TRUE OR FALSE

Without a space in front, GPT-2 splits *strawberry* into *st* + *raw* + *berry*. "Now the model can see each letter."

True ☐ False ☐

#### 7.4 TRUE OR FALSE

"Token 3798 must mean something close to `▮cat` (3797), because the IDs are neighbours."

True ☐ False ☐

## ANSWERS

# Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

### 1. Text in, numbers out: the tokenizer • rewatch ▶ 0:08-0:29

**1.1 False.** It reads a list of numbers, the token IDs produced by the tokenizer.

**1.2 B.** Predicting and picking are the model's loop from episode 1. The tokenizer only converts between text and numbers, in both directions.

**1.3 The tokenizer turns 262 back into text: *the*.** Output goes through the tokenizer too, in the reverse direction. In the video's list, 262 is the last token, *the*.

### 2. Two extremes: characters or words • rewatch ▶ 0:29-0:58

**2.1 22 and 6.** 17 letters + 5 spaces = 22 characters, against just 6 words.

**2.2 B.** A word tokenizer only knows the whole words in its fixed vocabulary. Splitting a word into pieces is exactly what it can't do.

**2.3 True.** Every word is built from characters the vocabulary already has. The price is long sequences with little meaning per token.

**2.4 C.** A, B and D are all word-tokenizer problems.

### 3. Subword tokens: the middle road • rewatch ▶ 0:58-1:16

**3.1 *the* is very common; *tokenization* is rarer.** Frequent words earn their own token. Rarer words are built from pieces that are themselves common.

**3.2 B.** It becomes *cat* + *flu* + *encer*. With subwords, no word is unreadable.

**3.3 6 and 21.**  $1 + 2 + 3 = 6$  subword tokens against  $3 + 12 + 6 = 21$  characters.

**3.4 False.** Only common words are one token. *tokenization* takes 2 and *catnap* takes 3.

### 4. Byte pair encoding: merge the most frequent pair • rewatch ▶ 1:16-1:50

**4.1 start with characters → count pairs → merge the most frequent → repeat.** Each round counts the pairs in the current text and merges the winner.

**4.2 u + g, 3 times.** It appears in hug, pug and hugs. h+u and p+u appear twice; g+s and u+n once.

**4.3 s + low + est.** s l o w e s t → s l o w e s t → s l o w e s t → s l o w e s t → s l o w e s t.

**4.4 False.** A merge adds a token; nothing is removed. e is still needed, e.g. in *n e w e s t*, where it is not followed by s.

## 5. BPE training in code · rewatch ► 1:50-2:01

**5.1 B.** After  $e+s$  merges, the pair  $es+t$  exists for the first time, and it wins the next round.

**5.2 13.** 9 starting characters + 1 new token per merge =  $9 + 4$ .

**5.3 (a) 10 pairs; ('e', 's') → 3 · (b) [('e', 's'), ('es', 't'), ('l', 'o'), ('lo', 'w')] · (c) ('low', 'est')** (a) `def count_pairs(words): return Counter(p for w in words for p in zip(w, w[1:]))`. The 10 pairs are the 5 in the video's table plus the 5 “seen once”. (b) The same four merges as the video. When counts tie ( $e+s$  and  $s+t$  both 3), `max` returns the pair it counted first. (c) After four merges every remaining pair appears only once, so it is a 7-way tie and `max` takes the first pair counted,  $low+est$  (from *lowest*). With a huge training text, exact ties like this are rare.

## 6. Vocabulary and token IDs · rewatch ► 2:01-2:22

**6.1 “The sat on thecat”.** 9246 is *cat* with no space in front, so it sticks to *the*. With 3797 (`␣cat`) it would read “... the cat”.

**6.2 50,257.**  $256 + 50,000 + 1 = 50,257$ : one token per starting symbol, one per merge, plus the special token.

**6.3 B.** At the start of “*cat food*” there is no space, so it is *cat* (9246); in “*my cat*” it is `␣cat` (3797).

## 7. The model never sees letters · rewatch ► 2:22-2:50

**7.1 B.** The letters are inside a single token; the model only gets the number 41236.

**7.2 8.** How · `␣many` · `␣r` · 's · `␣are` · `␣in` · `␣strawberry` · ?: about 4 characters per token.

**7.3 False.** It sees three IDs instead of one. The r's are still hidden, inside *raw* and *berry*.

**7.4 False.** An ID is just a label. (Episode 3 shows that 3798 is `esc`.)